

Module 2: Syntax Analysis

Context-Free Grammars, LL(1), SLR(1), CLR(1), LALR(1) & Parsing Conflicts

Module II: Syntax Analysis — Topics Covered

1. Role of Parser & Context-Free Grammars (CFG)
2. Ambiguity & Derivations (LM / RM)
3. Grammar Rewriting (Left Recursion & Factoring)
4. Top-Down Parsing: LL(1) Table Construction
5. Computation of FIRST and FOLLOW Sets
6. Bottom-Up Parsing & Handle Pruning
7. LR Parsers: LR(0), SLR(1), CLR(1), LALR(1)
8. Parsing Conflicts & Error Recovery Strategies

1. Context-Free Grammars (CFG) & Ambiguity

A **Context-Free Grammar** is a 4-tuple $G = (V, \Sigma, R, S)$ where V is a finite set of non-terminal variables, Σ is a finite set of terminal symbols ($V \cap \Sigma = \emptyset$), R is a finite set of production rules of the form $A \rightarrow \alpha$ ($A \in V, \alpha \in (V \cup \Sigma)^*$), and $S \in V$ is the start symbol.

Leftmost vs. Rightmost Derivations

Leftmost Derivation (LMD): At each step, the leftmost non-terminal is replaced first.

Rightmost Derivation (RMD / Canonical Derivation): At each step, the rightmost non-terminal is replaced first.

Ambiguous Grammar: A grammar that produces *two or more distinct parse trees* (or distinct LMDs) for at least one string $w \in L(G)$. (Classic example: "Dangling-Else" ambiguity).

2. Grammar Transformations for Top-Down Parsing

2.1 Elimination of Immediate Left Recursion

A production rule $A \rightarrow A\alpha \mid \beta$ is immediately left-recursive. It causes top-down predictive parsers to enter an infinite loop. We eliminate it by introducing a new non-terminal A' :

$$A \rightarrow \beta A', \quad A' \rightarrow \alpha A' \mid \epsilon$$

2.2 Left Factoring (Resolving Common Prefixes)

When two productions for A share a common leading prefix $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$, a predictive parser cannot decide which production to choose based on 1 lookahead token. We left-factor:

$$A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2$$

3. Computation of FIRST and FOLLOW Sets

Set	Mathematical Definition	Construction Rules
$\text{FIRST}(\alpha)$	$\text{FIRST}(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta\}$ (includes ϵ if $\alpha \Rightarrow^* \epsilon$)	1. If $X \in \Sigma$, $\text{FIRST}(X) = \{X\}$. 2. If $X \rightarrow \epsilon$, add $\epsilon \in \text{FIRST}(X)$. 3. For $X \rightarrow Y_1 Y_2 \dots Y_k$: add $\text{FIRST}(Y_1) \setminus \{\epsilon\}$. If $Y_1 \Rightarrow^* \epsilon$, add $\text{FIRST}(Y_2) \setminus \{\epsilon\}$, continuing until a non-nullable Y_i is reached. If all $Y_i \Rightarrow^* \epsilon$, add ϵ .
$\text{FOLLOW}(A)$	$\text{FOLLOW}(A) = \{a \in \Sigma \cup \{\$\} \mid S \Rightarrow^* \alpha A a \beta\}$	1. Add $\$$ to $\text{FOLLOW}(S)$ (where S is start symbol). 2. If $A \rightarrow \alpha B \beta$, add $\text{FIRST}(\beta) \setminus \{\epsilon\}$ to $\text{FOLLOW}(B)$. 3. If $A \rightarrow \alpha B$ or $A \rightarrow \alpha B \beta$ where $\epsilon \in \text{FIRST}(\beta)$, add all elements of $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

4. Top-Down LL(1) Parsing Table Construction

For each production $A \rightarrow \alpha$:

- For every terminal $a \in \text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
- If $\epsilon \in \text{FIRST}(\alpha)$, then for every terminal $b \in \text{FOLLOW}(A)$ (including $\$$), add $A \rightarrow \alpha$ to $M[A, b]$.

LL(1) Condition: A grammar is LL(1) iff no cell in parsing table M contains multiple entries (no conflicts).

5. Bottom-Up LR Parsers (LR(0), SLR(1), CLR(1), LALR(1))

Parser Type	Item Representation	Lookahead & Table Size	Power & Conflict Resolution
LR(0)	$A \rightarrow \alpha \cdot \beta$	0 lookaheads. Same number of states as SLR(1).	Weakest; frequent Shift/Reduce conflicts on reduction.
SLR(1)	$A \rightarrow \alpha \cdot \beta$	Place reduction $A \rightarrow \alpha$ only in $\text{FOLLOW}(A)$.	More powerful than LR(0), but still suffers conflicts on ambiguous grammars.
CLR(1)	$[A \rightarrow \alpha \cdot \beta, a]$ where $a \in \Sigma \cup \{\$\}$	Carries exact lookahead a . Huge state table.	Most powerful LR parser for deterministic CFGs. Large memory footprint.
LALR(1)	Merged CLR(1) items with identical core	Same number of states as SLR(1), lookaheads merged.	Standard in Yacc/Bison ; rarely introduces R/R conflicts, never introduces S/R conflicts.

BIT Mesra Exam Question & Solved Walkthrough (10 Marks)

Question: Construct the LL(1) parsing table for the grammar: $E \rightarrow TE', E' \rightarrow +TE' \mid \epsilon, T \rightarrow FT', T' \rightarrow *FT' \mid \epsilon, F \rightarrow (E) \mid \text{id}$.

Solution:

- $\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{(\text{, id}\}$
- $\text{FIRST}(E') = \{+, \epsilon\}, \text{FIRST}(T') = \{*, \epsilon\}$
- $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$\}$
- $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{+,), \$\}$
- $\text{FOLLOW}(F) = \{*, +,), \$\}$

Every cell in table M contains at most one production $\implies$ The grammar is strictly LL(1).